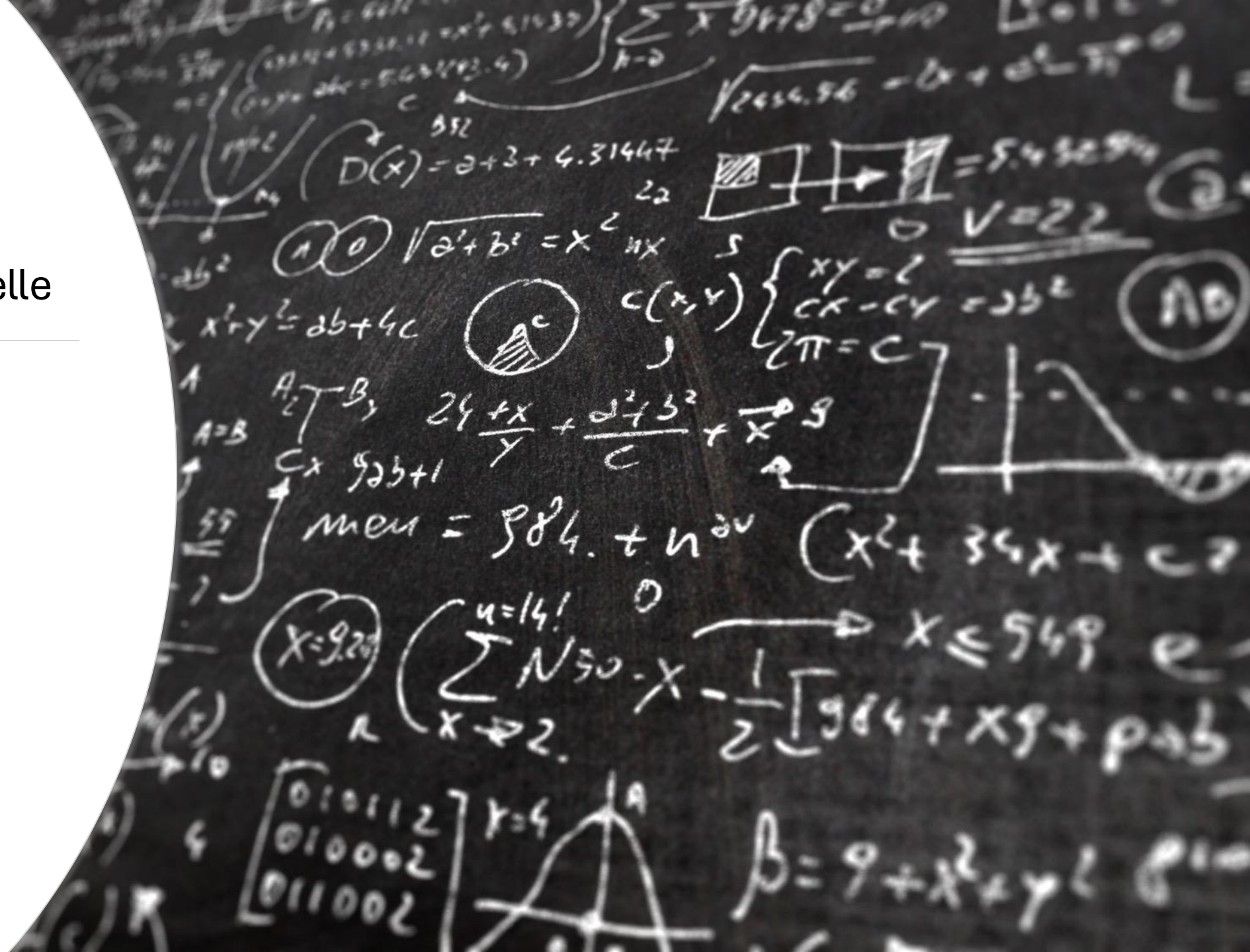


Introduction à l'Intelligence Artificielle

Jean-Jacques Bernard

La Dynamo – Chambéry



Ce que nous allons couvrir aujourd'hui

De quoi parle-t'on ?

Autant en parler tout de suite : ChatGPT et ses amis...

Vision par ordinateur...

... en pratique !

Et après ?

N'hésitez pas à poser vos questions au fur et à mesure !

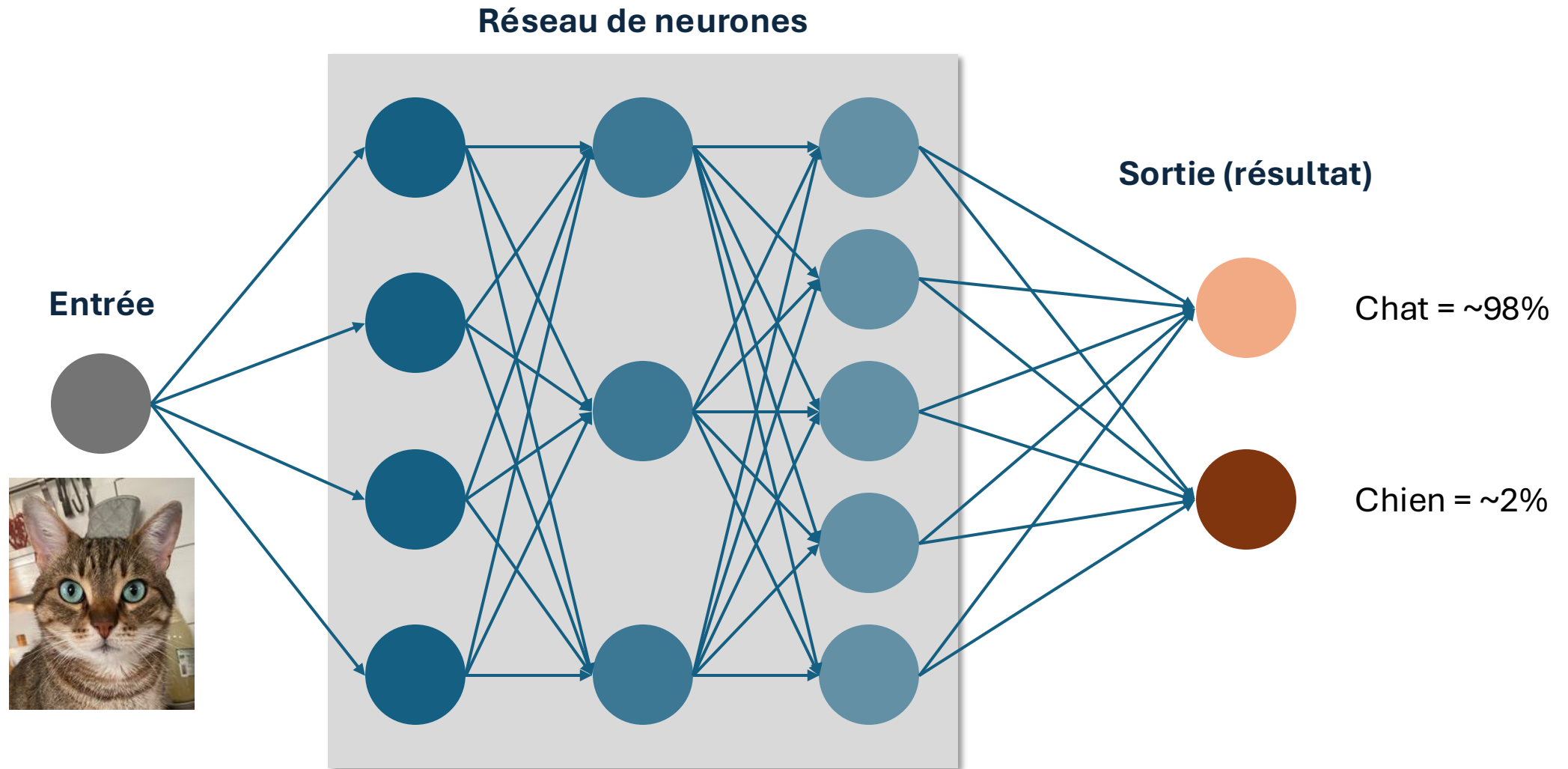
https://github.com/jjbernard/Formation_IA_Dynamo

Alors, l'IA c'est quoi ?

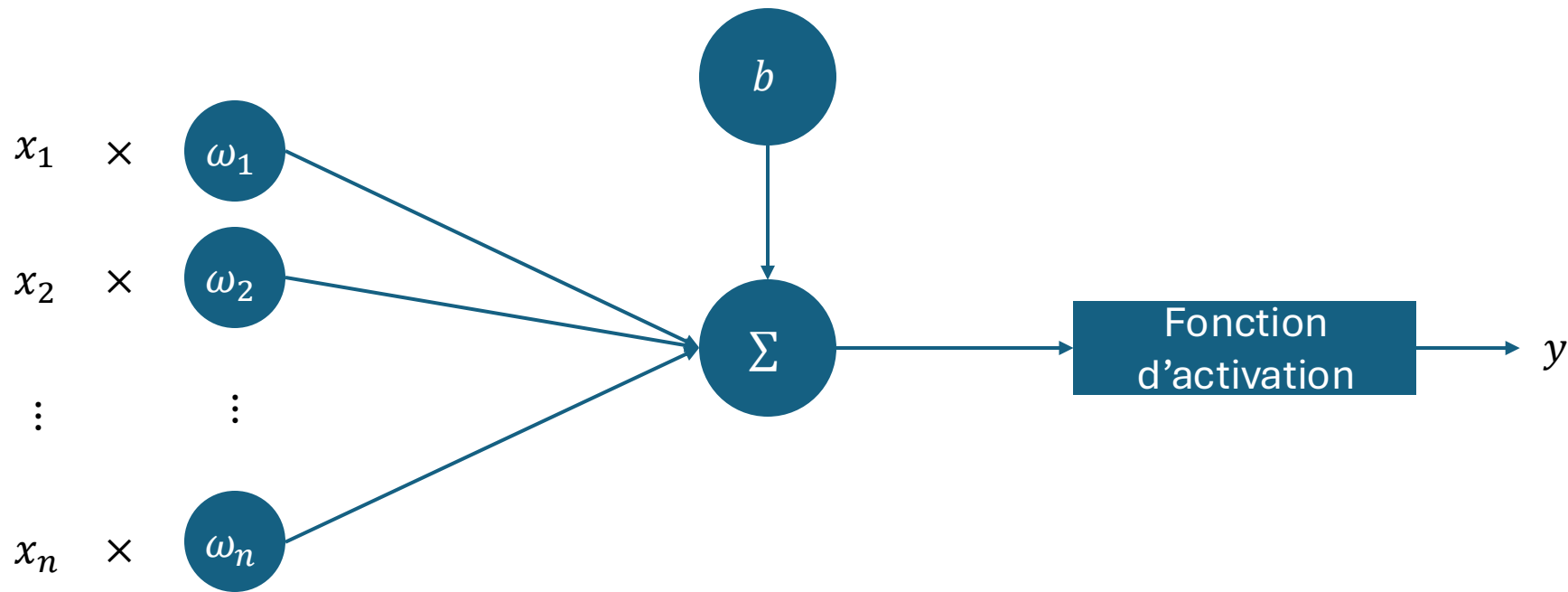
Beaucoup de termes se sont succédés ces dernières années :

- *Machine Learning* / Apprentissage automatique :
 - Pas de l'IA en tant que tel, mais c'est l'ancêtre
- Réseaux de neurones :
 - Les blocs de base de l'IA
- *Deep Learning* :
 - Un réseau de neurones... profond. C'est un peu du marketing mais le terme est resté
- *Generative AI* / IA Générative :
 - Un réseau de neurones basé sur une architecture permettant de générer du contenu

La brique de base : le réseau de neurones

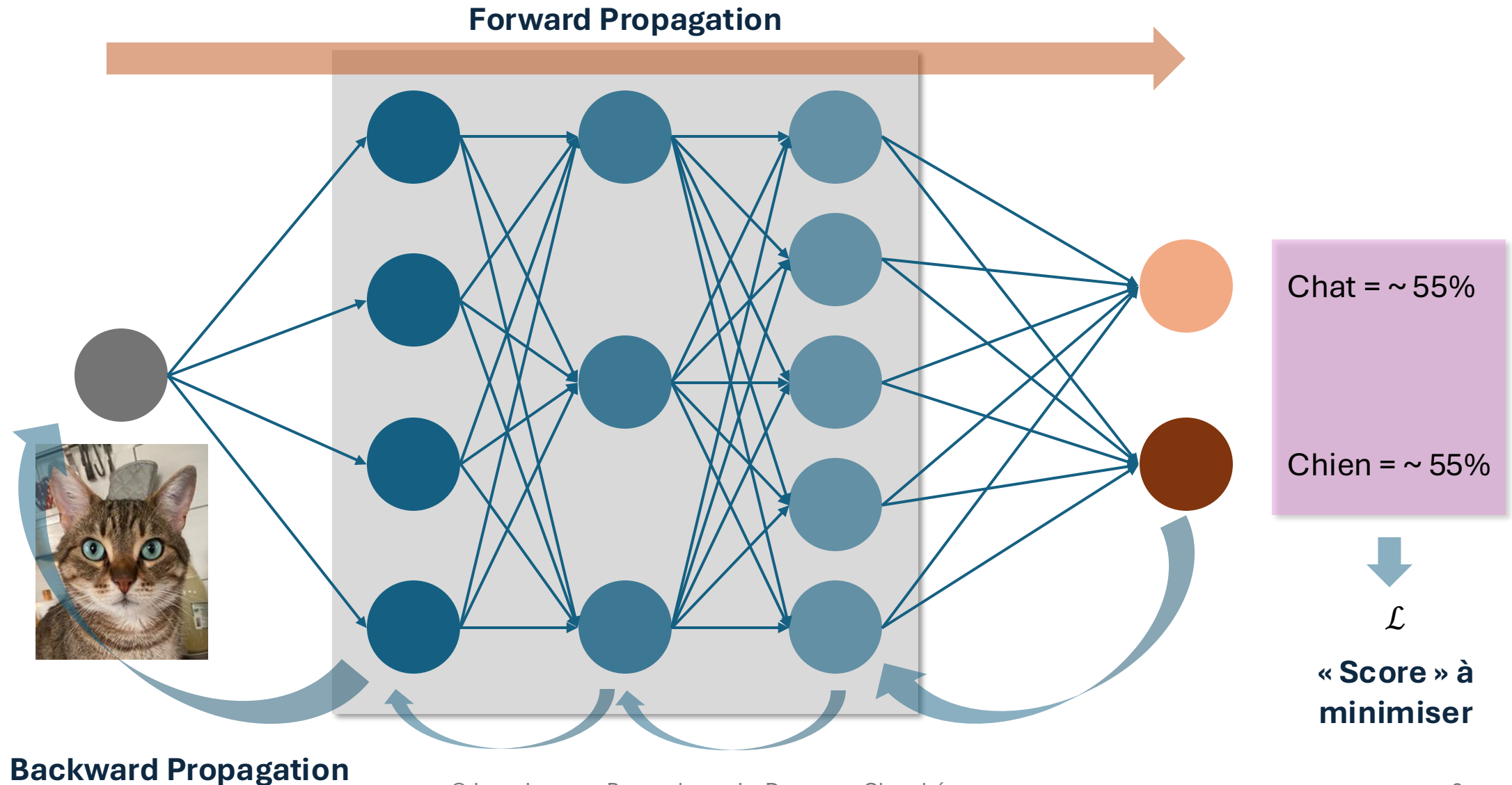


Qu'y a t'il dans un neurone ?



$$y = \omega x + b, \text{ tout simplement !}$$

La sauce secrète : propagation et « back propagation »



L'autre élément clé : les données pour « entraîner »

- En simplifiant, l'IA n'est « qu'un » gros (énorme) modèle probabiliste
- La seule chose que l'IA fait est de ressortir ce qui est de plus probable par rapport à ce que l'IA « sait »
- Comment l'IA « sait », justement ? Les données !!
- Enormément de données !!
 - ImageNet : 1 281 167 images pour l'entraînement → utilisé dans tous les modèles de vision par ordinateur
 - Common Crawl : 2,74 milliards de pages web
 - Etc.

Trouver et assembler des données (créer un *dataset*) est la partie la moins intéressante de l'IA, mais elle est absolument critique !

C'est quoi un GPT¹ ? (ce n'est pas mon chat 😊)

C'est un LLM : Large Language Model

Il y a 3 types de LLM, la majorité basée sur l'architecture « *transformer* » (image à droite) :

- Encodeurs :
 - Classification de texte
 - Recherche sémantique
 - Recherche d'entité

- Décodeurs
 - Génération de texte
 - Chatbot
 - Génération de code

LLM

- Encodeurs – Décodeurs
 - Traduction
 - Résumé
 - Paraphrase

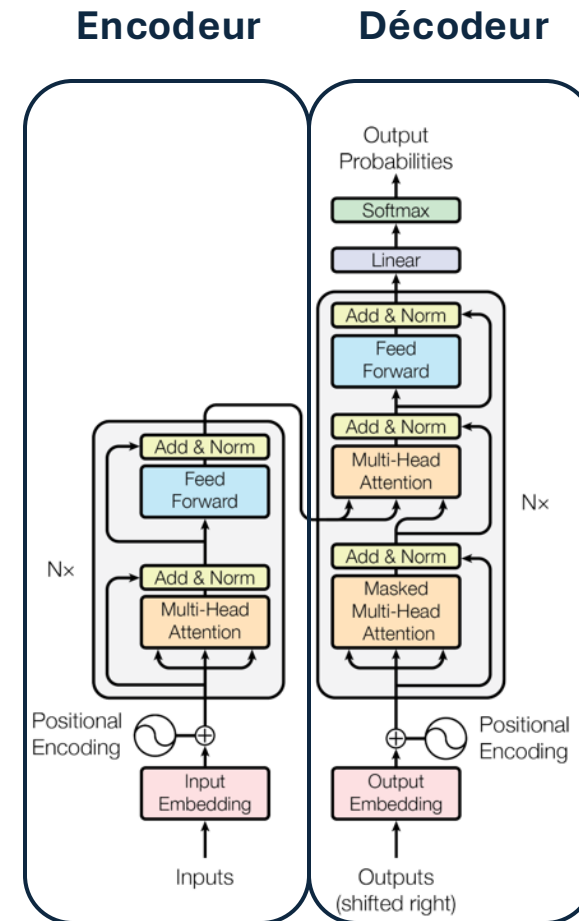


Figure 1: The Transformer - model architecture.

¹ GPT: Generative Pre-trained Transformer
Source : Attention Is All You Need <https://arxiv.org/abs/1706.03762>

Il existe beaucoup d'autres architectures, et ce n'est surement pas fini !

Exemple pour les images : modèles dit « de diffusion » comme Dall-E, Stable Diffusion, Black Forest Flux. Ces modèles sont basés sur un modèle d'autoencodage de l'information

Mais aussi des architectures optimisées pour la génération de texte : **Mamba**, une architecture focalisée sur l'efficacité

L'apprentissage par renforcement (*reinforcement learning*) :

- utilisé auparavant pour faire des super scores aux jeux vidéos,
- mais maintenant pour créer les modèles qui « raisonnent » (en sachant qu'il y a des techniques qui permettent de faire la même chose avec un modèle qui ne « raisonne » pas, telles que le concept de « Chaîne de Pensées »).

Une image c'est quoi ? (1/2)

3 canaux de couleur : RVB



Une image c'est quoi ? (2/2)

3 canaux de couleur : RVB



(0, 193, 0)



(217, 0, 0)



(0, 0, 174)



(217, 193, 174)

Les autres méthodes pour encoder une image

HLS

Utilisation en photographie

- H: *Hue* / Teinte
- L: *Lightness* / Clarté
- S: Saturation

HSV

Similaire à HLS

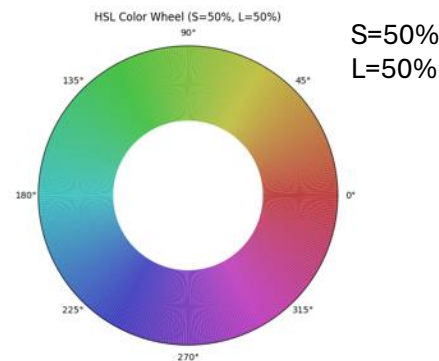
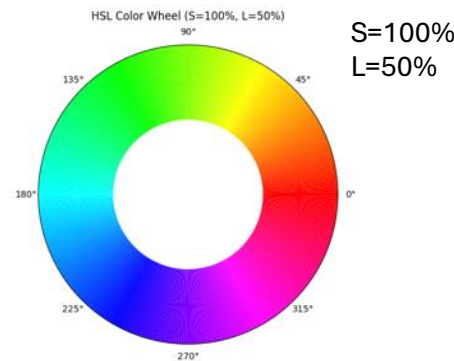
- V: *Value* / Valeur

CYMK

Utilisation en imprimerie

- C: Cyan
- Y: *Yellow* / Jaune
- M: Magenta
- K: *Key (Black)* / Noir

Roue des couleurs HSV



Ces différents systèmes d'encodage ne sont pas utilisés par les fichiers images qui reposent sur RGB principalement.

Par contre, ils sont utilisés par les logiciels de traitement d'image : GIMP, Affinity Photo, Photoshop, etc.

Et aussi par OpenCV...

Le cœur des modèles de vision par ordinateur : les CNN (*Convolutional Neural Networks*) (1/3)

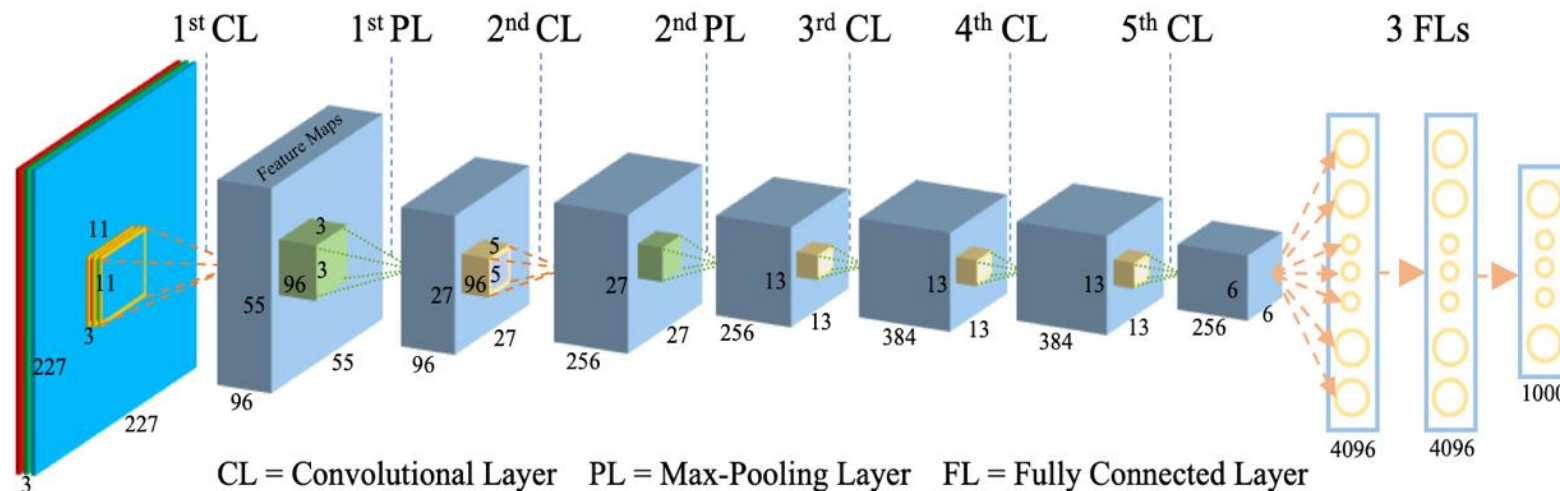
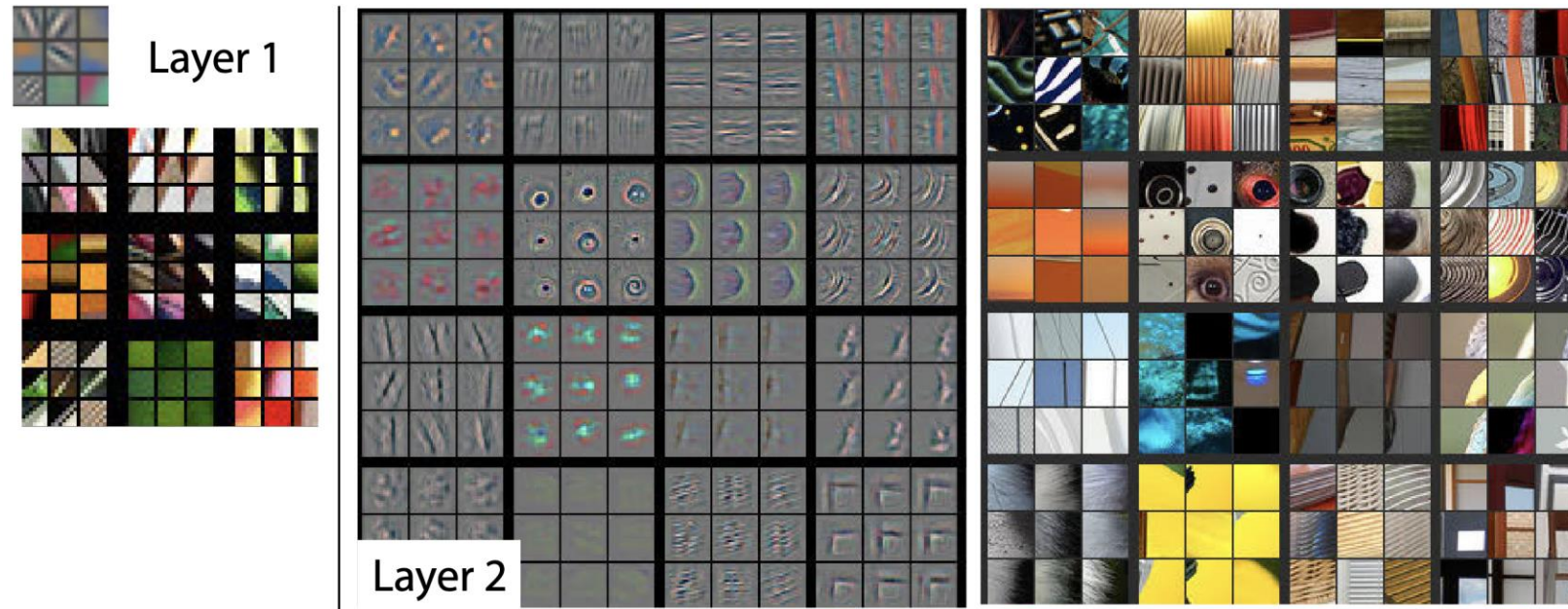


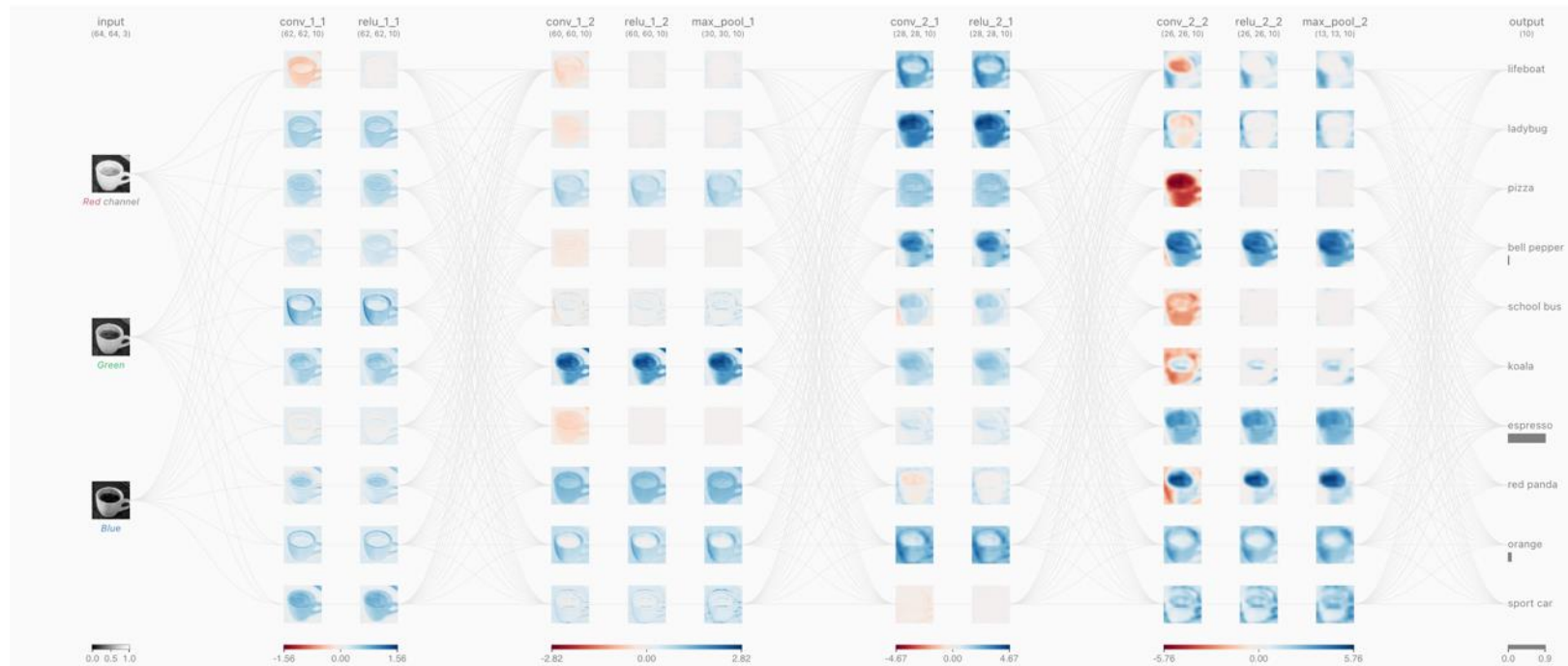
FIGURE 1. *CaffeNet* architecture

Le cœur des modèles de vision par ordinateur : les CNN (*Convolutional Neural Networks*) (2/3)



Source : "Visualizing and Understanding Convolutional Networks": <https://arxiv.org/abs/1311.2901>

Le cœur des modèles de vision par ordinateur : les CNN (*Convolutional Neural Networks*) (3/3)



Source : "CNN Explainer": <https://poloclub.github.io/cnn-explainer/>

Domaines d'application des CNN

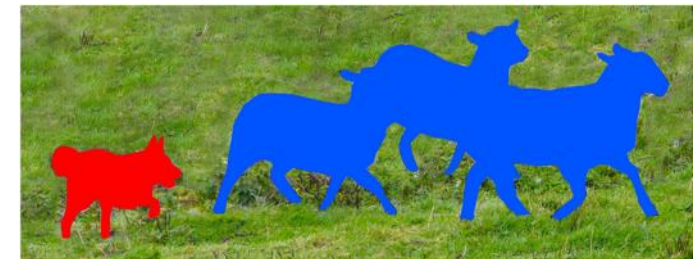
Les domaines (non exhaustif)

- La reconnaissance d'image / de vidéo
- La classification d'image
- La segmentation d'image
- La reconnaissance d'objets
- L'analyse d'images médicales
- Les séries temporelles (en finance)
- Le traitement du langage naturel
- Etc.

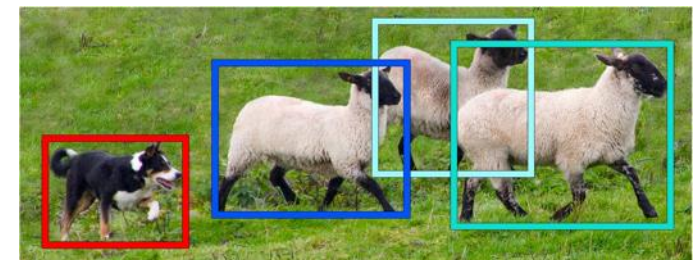
Classification



Segmentation



Detection d'objet



Source : Wikipedia: https://en.wikipedia.org/wiki/Convolutional_neural_network, MIT Robotic Manipulation <https://manipulation.csail.mit.edu/> & https://manipulation.csail.mit.edu/data/coco_instance_segmentation.jpeg

Les modèles de vision par ordinateur

Beaucoup de modèles sont disponibles, tous avec une spécialité potentiellement différente. Quelques exemples :

- ResNet : (Residual Neural Network) : classification
- VGGNets : classification
- MobileNet: classification et détection d'objets
- U-Net : segmentation
- SegNet : segmentation
- YoLo (You Look Only Once) : détection d'objets

```
>>> import torchvision
>>> models = torchvision.models.list_models()
>>> models
['alexnet', 'convnext_base', 'convnext_large', 'convnext_small',
', 'densenet121', 'densenet161', 'densenet169', 'densenet201', ...]
```

Comment on utilise tout cela ?

2 façons :

- Facile :
 - On reprend les modèles pré-entraînés dans les « *frameworks* » de « *deep learning* » ou de vision par ordinateur existants et on les utilise tels quels / on les « *fine tune* »
 - Ca permet de démarrer rapidement et de prototyper !
- Difficile :
 - On code les architectures à la main et on entraîne à partir de 0 ! 😅
 - C'est long, c'est compliqué, mais il n'y a pas mieux pour comprendre et apprendre !

⚠ Dans les deux cas... Attention à l'entraînement et au *fine tuning*, c'est long... voire très long ! Un ordinateur avec un GPU¹ est **INDISPENSABLE** ! ⚠

1 et de préférence un de chez Nvidia, oui c'est un quasi monopole !

Détection d'objet avec YoLo !



https://github.com/jjbernard/Formation_IA_Dynamo

Dataset : *litter-detection*

<https://www.kaggle.com/datasets/davianmartinovci/litter-detection?resource=download>

Source : réalisé avec Black Forest Labs Flux Pro à travers Le Chat de Mistral. "What is YOLOv5: A deep look into the internal features of the popular object detector" <https://arxiv.org/abs/2407.20892>

Quelques échelles de temps pour le *fine-tuning*

50 epochs avec yolov5 Large + freeze

```
50 epochs completed in 2.564 hours.
Optimizer stripped from runs/train/exp5/weights/last.pt, 92.9MB
Optimizer stripped from runs/train/exp5/weights/best.pt, 92.9MB

Validating runs/train/exp5/weights/best.pt...
Fusing layers...
Model summary: 267 layers, 46124433 parameters, 0 gradients, 107.7 GFLOPs

```

Class	Images	Instances	P	R	mAP50	mAP50-95
all	373	953	0.946	0.931	0.954	0.846
Metal	373	187	0.994	0.947	0.97	0.935
Cardboard	373	244	1	0.965	0.984	0.868
Glass	373	396	0.984	0.987	0.989	0.892
Plastic	373	126	0.807	0.825	0.874	0.69

Results saved to runs/train/exp5

80 epochs avec yolov5 small

```
80 epochs completed in 2.210 hours.
Optimizer stripped from runs/train/exp10/weights/last.pt, 14.5MB
Optimizer stripped from runs/train/exp10/weights/best.pt, 14.5MB

Validating runs/train/exp10/weights/best.pt...
Fusing layers...
Model summary: 157 layers, 7020913 parameters, 0 gradients, 15.8 GFLOPs

```

Class	Images	Instances	P	R	mAP50	mAP50-95
all	373	953	0.938	0.912	0.94	0.817
Metal	373	187	0.997	0.93	0.958	0.923
Cardboard	373	244	0.97	0.937	0.972	0.818
Glass	373	396	0.992	0.97	0.989	0.877
Plastic	373	126	0.794	0.81	0.839	0.651

Results saved to runs/train/exp10

50 epochs avec yolov5 Medium + freeze 10

```
50 epochs completed in 1.629 hours.
Optimizer stripped from runs/train/exp7/weights/last.pt, 42.2MB
Optimizer stripped from runs/train/exp7/weights/best.pt, 42.2MB

Validating runs/train/exp7/weights/best.pt...
Fusing layers...
Model summary: 212 layers, 20865057 parameters, 0 gradients, 47.9 GFLOPs

```

Class	Images	Instances	P	R	mAP50	mAP50-95
all	373	953	0.945	0.924	0.95	0.806
Metal	373	187	0.983	0.932	0.955	0.901
Cardboard	373	244	0.975	0.942	0.985	0.816
Glass	373	396	0.96	0.981	0.992	0.862
Plastic	373	126	0.86	0.841	0.869	0.646

Results saved to runs/train/exp7

Matériel utilisé : VM avec un Nvidia Quadro P4000 (~ 2017) → pas trop cher !



30 post training après les 50 epochs d'avant

```
30 epochs completed in 1.826 hours.
Optimizer stripped from runs/train/exp8/weights/last.pt, 42.2MB
Optimizer stripped from runs/train/exp8/weights/best.pt, 42.2MB

Validating runs/train/exp8/weights/best.pt...
Fusing layers...
Model summary: 212 layers, 20865057 parameters, 0 gradients, 47.9 GFLOPs

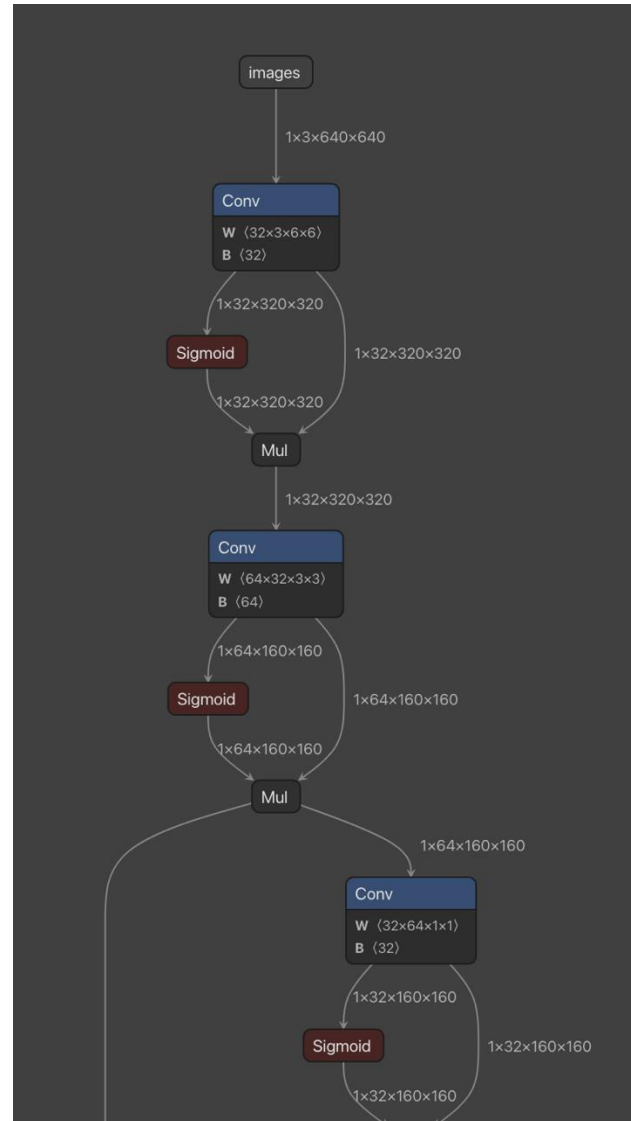
```

Class	Images	Instances	P	R	mAP50	mAP50-95
all	373	953	0.963	0.938	0.958	0.851
Metal	373	187	0.984	0.936	0.958	0.926
Cardboard	373	244	0.992	0.971	0.988	0.873
Glass	373	396	0.982	0.99	0.994	0.908
Plastic	373	126	0.892	0.853	0.892	0.695

Results saved to runs/train/exp8

Pour visualiser le contenu d'un modèle

<https://netron.app>



L'IA est un sujet en pleine expansion, avec **beaucoup** (trop ?) de sujets à creuser !

Quelques sujets d'exploration (liste non exhaustive bien sur !)

- Optimisation des architectures et de l'entraînement (quantisation, transfer learning, etc.)
- Fine tuning de LLM / SLM / modèles de diffusion
- Utilisation locale sur des périphériques contraints (Raspberry Pi, Arduino...) soit par optimisation soit à travers TinyML (ou les deux)
- Application à la robotique
- Cas d'usage de l'IA générative au delà des Chatbot et de la génération d'images
- Agents IA
- Etc.